PROJECT TECHNICAL REPORT & FLOW ANSWER SHEET

Design, Self-Checking Verification, and FPGA Implementation of a Resource-Shared 4-Bit Arithmetic Logic Unit

(ALU_4bit_SE190745)

Le Ky Dung – Roll Number: SE190745
Department of Microelectronics and IC Design, FPT University Ho Chi Minh City

October 2026

Abstract

This technical report presents the complete Register-Transfer Level (RTL) design, self-checking testbench verification, simulation truth/step table, and step-by-step physical implementation flow (documented via full-window screenshots F1 through F6) for a resource-shared 4-bit Arithmetic Logic Unit (alu_4bit) developed in AMD Xilinx Vivado 2026.1 targeting the Artix-7 FPGA (xc7a100tcsg324-1). The architecture executes ten distinct operations specified by a 4-bit opcode (opcode[3:0]): single-cycle combinational addition, two's complement subtraction, bitwise logic (AND, OR, XOR, NOT), logical shifts (SHL, SHR), and 4-cycle sequential multiplication and restoring division. By dynamically sharing a single 4-bit ripple-carry adder/subtractor core between combinational instructions and the sequential multiplier/divider finite state machine (muldiv_unit) via 2-to-1 multiplexers controlled by busy, the post-implementation design achieves an ultra-compact footprint of only 60 Slice LUTs (0.09%), 26 Slice Flip-Flops (0.02%), 30 Bonded IOBs (14.29%), and 1 BUFG (3.13%).

---

1 Part A & Part B: Modular RTL Architecture and Self-Checking Testbench

1.1 Hierarchical RTL Structure and Shared-Adder Datapath

As illustrated in the full-window Project Manager view in Figure 1 (F1.png), the project ALU_4bit_SE190745 is organized hierarchically under the top-level module alu_4bit.v, which integrates six specialized functional submodules (ctrl, adder_sub_u, logic_u, shifter_u, muldiv_u, and flags_u). All source files include standardized header comments specifying the project name, author (Le Ky Dung), and roll number (SE190745).

To optimize silicon area, muldiv_unit does not instantiate a dedicated adder. Instead, three 2-to-1 multiplexers (mux_adder_a_i, mux_adder_b_i, and mux_adder_sub_i) controlled by the busy signal route external operands (a[3:0], b[3:0], sub) to adder_subtractor_4bit when busy $= 0$, and switch to the internal sequential accumulator paths (add_a[3:0], add_b[3:0], is_div) during 4-cycle multiplication or division (busy $= 1$).

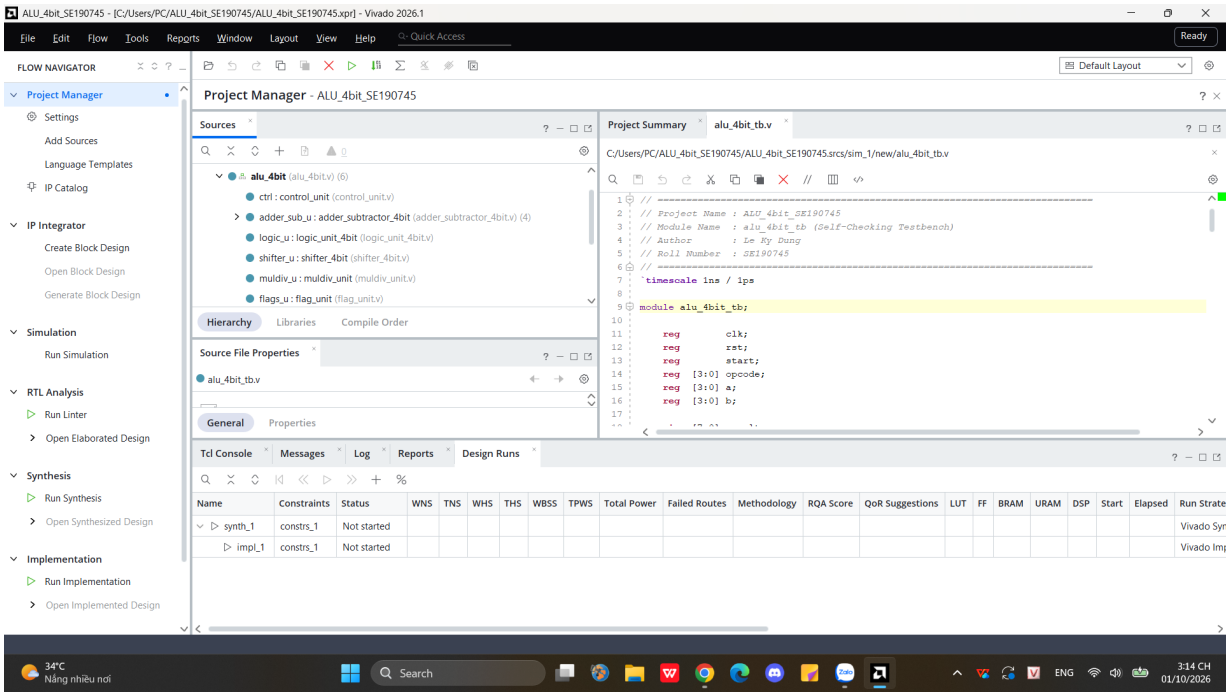


Figure 1: **Flow Screenshot F1:** Full-window Vivado Project Manager view showing the visible project name (`ALU_4bit_SE190745`), complete 6-submodule RTL hierarchy under `alu_4bit`, and the self-checking testbench (`alu_4bit_tb.v`) with author header comments.

1.2 Submodule Equations and Elaborated Schematic (F2)

1. **Ripple-Carry Adder/Subtractor** (`adder_subtractor_4bit.v` & `full_adder.v`)**:** Chains four 1-bit full adders (`fa0`–`fa3`) computing $\text{sum}_i = a_i \oplus b_{\text{inv},i} \oplus c_i$ and $c_{i+1} = (a_i \cdot b_{\text{inv},i}) + (a_i \oplus b_{\text{inv},i}) \cdot c_i$, where $b_{\text{inv}}[i] = b[i] \oplus \texttt{sub}$ and $c_0 = \texttt{sub}$ to perform two's complement subtraction ($A - B = A + \overline{B} + 1$).

2. **Opcode Control Decoder** (`control_unit.v`)**:** Decodes `opcode[3:0]` into six active-high control lines:

$$\begin{aligned} \texttt{sub} &= \overline{\text{op}[3]} \cdot \overline{\text{op}[2]} \cdot \overline{\text{op}[1]} \cdot \text{op}[0] \ (0001), & \texttt{arith} &= \overline{\text{op}[3]} \cdot \overline{\text{op}[2]} \cdot \overline{\text{op}[1]} \ (000x) \\ \texttt{is_shl} &= \overline{\text{op}[3]} \cdot \text{op}[2] \cdot \text{op}[1] \cdot \overline{\text{op}[0]} \ (0110), & \texttt{is_shr} &= \overline{\text{op}[3]} \cdot \text{op}[2] \cdot \text{op}[1] \cdot \text{op}[0] \ (0111) \\ \texttt{is_div} &= \text{op}[3] \cdot \overline{\text{op}[2]} \cdot \overline{\text{op}[1]} \cdot \text{op}[0] \ (1001), & \texttt{is_muldiv} &= \text{op}[3] \cdot \overline{\text{op}[2]} \cdot \overline{\text{op}[1]} \ (100x) \end{aligned} \tag{1}$$

3. **Bitwise Logic** (`logic_unit_4bit.v`) **& 1-Bit Shifter** (`shifter_4bit.v`)**:** Computes `a & b`, `a | b`, `a ^ b`, `~a`, left shift $\texttt{out_shl} = \{a[2:0], 1'b0\}$, and right shift $\texttt{out_shr} = \{1'b0, a[3:1]\}$.

4. **Sequential Mul/Div Engine** (`muldiv_unit.v`)**:** Uses a 2-bit cycle counter (`count[1:0]` from 0 to 3), 4-bit registers (`acc`, `q`, `m`), and output latches $\{\texttt{hi[3:0]}, \texttt{lo[3:0]}\}$ to execute 4-cycle shift-and-add multiplication (`opcode = 8`) or 4-cycle restoring division (`opcode = 9`). If division by zero ($b =$ `4'b0000`) is requested, it immediately asserts `div_by_zero` $= 1$ and `done` $= 1$ without entering the calculation loop.

5. **Status Flag Unit** (`flag_unit.v`)**:** Computes Carry (`C`), Zero (`Z`), Overflow (`V`), and Negative (`N`):

$$\texttt{overflow} = \texttt{arith} \cdot (c_3 \oplus c_4), \qquad \texttt{carry} = (\texttt{arith} \cdot c_4) + (\texttt{is_shl} \cdot a[3]) + (\texttt{is_} \tag{2}$$

$$\texttt{zero} = \texttt{is_muldiv} \ ? \ \overline{(|\texttt{result[7:0]})} : \overline{(|\texttt{result[3:0]})}, \quad \texttt{negative} = \texttt{is_muldiv} \ ? \ 1'b0 : \texttt{result[3]} \tag{3}$$

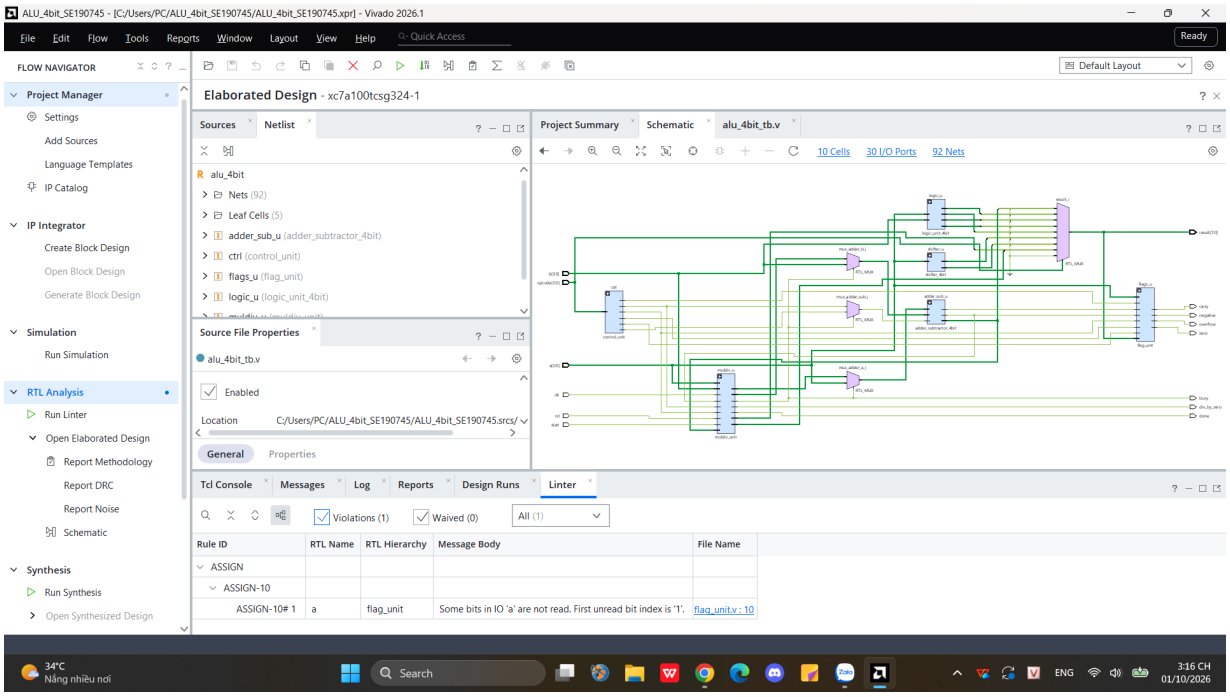


Figure 2: **Flow Screenshot F2:** Full-window Elaborated RTL Schematic of `ALU_4bit_SE190745` showing 10 top-level cells (6 submodules + 4 RTL multiplexers), 30 I/O ports, 92 nets, and static Linter rule `ASSIGN-10#1`.

2 Part C: Simulation Truth / Step Table and Waveform Analysis

The self-checking testbench (`alu_4bit_tb.v`) verifies both **Part 1** (combinational opcodes 0–7) and **Part 2** (sequential opcodes 8–9 and divide-by-zero exception), maintaining an internal 32-bit hardware error counter (`error_count[31:0]`) and printing a final `PASS` confirmation. Table 1 documents the exact Truth / Step Table extracted directly from the simulation waveforms in Figures 3 and 4.

Table 1: **Part C – Truth / Step Table:** Extracted Directly from Simulation Waveforms (F3 & F4)

Time (ns)	Opcode	Binary	Operation	a (Dec)	b (Dec)	`result[7:0]` (Hex)	C	Z	V	N	`error_count`	Waveform Verification Notes
$0-20$	0	0000	**RESET** (`rst` $=1$)	0	0	00	0	1	0	0	00000000	Initializes all registers to zero
$20-40$	0	0000	**ADD** ($a+b$)	5	3	08 (8)	0	0	1	1	00000000	Cursor at 20 ns; signed 4-bit overflow ($+5+3=-8$)
$40-60$	1	0001	**SUB** ($a-b$)	9	4	05 (5)	1	0	1	0	00000000	$c_4=1$ (no borrow); signed underflow ($-7-4=-11$)
$60-80$	2	0010	**AND** (a & b)	12	10	08 (8)	0	0	0	1	00000000	1100 & 1010 = 1000 (`result[3]` $=1$)
$80-100$	3	0011	**OR** ($a \mid b$)	12	3	0f (15)	0	0	0	1	00000000	$1100 \mid 0011 = 1111$ (`result[3]` $=1$)
$100-120$	4	0100	**XOR** ($a \oplus b$)	12	10	06 (6)	0	0	0	0	00000000	$1100 \oplus 1010 = 0110$
$120-140$	5	0101	**NOT** ($\sim a$)	10	10	05 (5)	0	0	0	0	00000000	$\sim 1010 = 0101$ (unary, ignores $b=10$)
$140-160$	6	0110	**SHL** ($a \ll 1$)	3	10	06 (6)	0	0	0	0	00000000	$0011 \ll 1 = 0110$, `carry` $= a[3] = 0$
$160-180$	7	0111	**SHR** ($a \gg 1$)	12	10	06 (6)	0	0	0	0	00000000	$1100 \gg 1 = 0110$, `carry` $= a[0] = 0$
$180-235$	8	1000	**MUL** ($a \times b$)	7	6	2a (42)	0	0	0	0	00000000	`busy` $=1$ (185–225 ns), `done` $=1$ at 225 ns
$245-305$	9	1001	**DIV** ($a \div b$)	13	3	14 (R $=1$, Q $=4$)	0	0	0	0	00000000	`busy` $=1$ (255–295 ns), `done` $=1$ at 295 ns
$305-385$	9	1001	**DIV** ($a \div 0$)	8	0	Holds 14	0	0	0	0	00000000	`div_by_zero` $=1$, `done` $=1$ at 315 ns

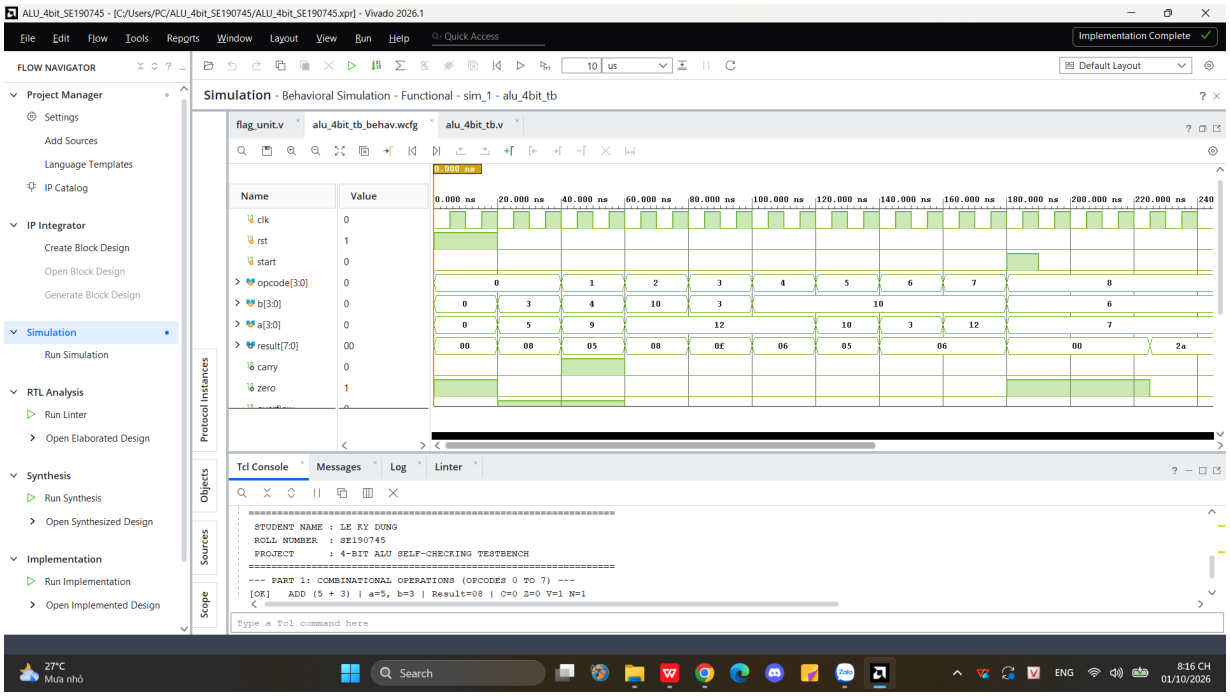


Figure 3: **Flow Screenshot F3:** Full-window Behavioral Simulation waveform for **Part 1** (0 ns to 220 ns, opcodes 0–7) with yellow cursor at 20.000 ns ($a = 5, b = 3$, `result` = `08`) and self-checking `error_count[31:0]` = `00000000`.

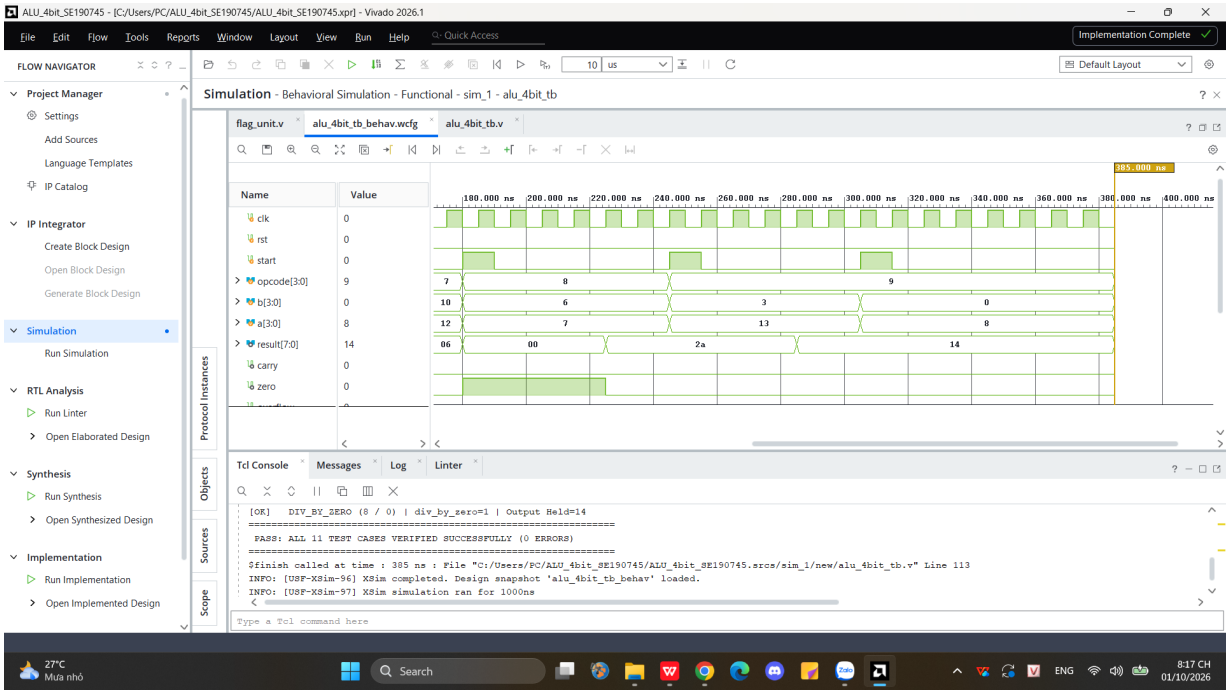


Figure 4: **Flow Screenshot F4:** Full-window Behavioral Simulation waveform for **Part 2** (180 ns to 385 ns) verifying 4-cycle multiplication ($7 \times 6 = 2a_{16} = 42$ at 225 ns), restoring division ($13 \div 3 = 14_{16} \rightarrow$ Rem 1, Quot 4 at 295 ns), divide-by-zero flag assertion ($8 \div 0 \rightarrow$ `div_by_zero` = 1 at 315 ns), and `error_count` = `00000000`.

3 Part D & Part E: Step-by-Step FPGA Design Flow Explanation

Each stage of the Vivado 2026.1 design and implementation flow (screenshots F1–F6) is explained below in detail:

1. **Step 1 – Project Creation & Hierarchical RTL Entry** (F1)**:** The project ALU_4bit_SE190745 was configured for the Artix-7 xc7a100tcsg324-1 package. Eight synthesizable Verilog modules were written and linked under alu_4bit, while alu_4bit_tb.v was placed under Simulation Sources. Vivado's hierarchy parser confirmed zero syntax errors ('0' warnings/errors in the Sources panel).

2. **Step 2 – RTL Elaboration & Static Linter Verification** (F2)**:** Running *Open Elaborated Design* generated the technology-independent schematic containing 10 top-level cells ('5' leaf cells/gates, '4' RTL multiplexers, and submodule blocks), 30 I/O ports, and 92 nets. The static Linter reported a single expected notification (ASSIGN-10#1 on flag_unit.v:10: *"Some bits in IO 'a' are not read. First unread bit index is '1"'*) because the Carry flag logic only reads a[3] (for SHL) and a[0] (for SHR), leaving middle bits a[1] and a[2] unconnected inside flag_unit.

3. **Step 3 – Behavioral Simulation of Combinational Instructions** (F3)**:** In XSim, opcode[3:0], b[3:0] (row 5), and a[3:0] (row 6) were displayed in *Unsigned Decimal*, while result[7:0] was displayed in *Hexadecimal*. Following reset release at 20 ns, opcodes 0 through 7 executed instantaneously without clock latency. At 20 ns (ADD : $5+3=8$) and 40 ns (SUB : $9-4=5$), overflow $=1$ asserted accurately because in 4-bit two's complement signed arithmetic ($[-8,+7]$), $+5+(+3)=+8$ overflows into -8 (1000), and 9 (1001, i.e., -7) minus $+4$ equals -11, underflowing -8.

4. **Step 4 – Behavioral Simulation of Sequential Mul/Div & Exception Handling** (F4)**:** At 180 ns, pulsing start with opcode $=8$ ($a=7, b=6$) asserted busy $=1$ from 185 ns to 225 ns (4 clock cycles); at 225 ns, done $=1$ pulsed and result updated to 8'h2a ($2\times16+10=42$). At 245 ns, pulsing start with opcode $=9$ ($a=13, b=3$) asserted busy $=1$ from 255 ns to 295 ns, outputting 8'h14 at 295 ns (Remainder hi $=$ 4'h1, Quotient lo $=$ 4'h4). Finally, testing $8\div0$ at 305 ns bypassed calculation (busy $=0$), asserted div_by_zero $=1$ and done $=1$ at 315 ns, and preserved the previous output 8'h14. Throughout all 11 test cases, error_count[31:0] remained strictly at 00000000.

5. **Step 5 – Synthesis & Post-Implementation Resource and Power Analysis** (F5)**:** Following synthesis and physical implementation, the Project Summary Overview (Table mode) confirmed that the entire ALU utilizes only **60 Slice LUTs** (0.09%), **26 Flip-Flops** (0.02%), **30 Bonded IOBs** (14.29%), and **1 BUFG** (3.13%). Because the IP core was evaluated without an external clock constraint file (.xdc), Vivado reported timing as NA and computed a vectorless unconstrained on-chip power estimate of 3.962 W (junction temperature 43.1°C, thermal margin 41.9°C / 9.1 W) at a *Low* confidence level.

6. **Step 6 – Physical Place-and-Route Verification on FPGA Die** (F6)**:** Opening the Implemented Design *Device* view verified that all logic cells and routing nets were physically placed inside Clock Region X0Y1 of the xc7a100tcsg324-1 die, specifically concentrated around Configurable Logic Block tile CLBLM_R_X3Y63 (Row 142, Column 13, encompassing slices SLICE_X2Y63 and SLICE_X3Y63) adjacent to the I/O bank column.

Table 2: Post-Implementation Hardware Resource Utilization on Artix-7 (xc7a100tcsg324-1) from F5

Resource Type	**Utilization (Used)**	**Available**	**Utilization (%)**
LUT (Slice Lookup Tables)	60	63,400	0.09%
FF (Slice Flip-Flop Registers)	26	126,800	0.02%
IO (Bonded Input/Output Pins)	30	210	14.29%
BUFG (Global Clock Buffers)	1	32	3.13%

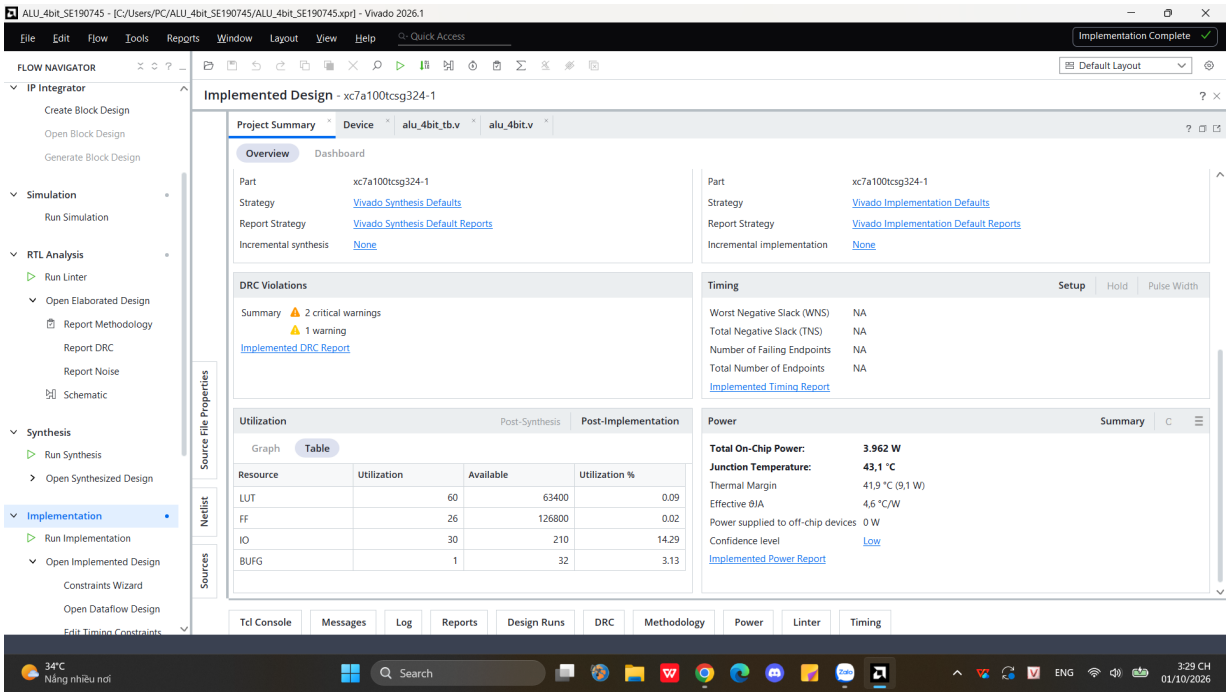


Figure 5: **Flow Screenshot F5:** Full-window Post-Implementation Project Summary showing the Utilization Table (60 LUTs, 26 FFs, 30 IOs, 1 BUFG) and Unconstrained Power Summary (3.962 W, 43.1°C).

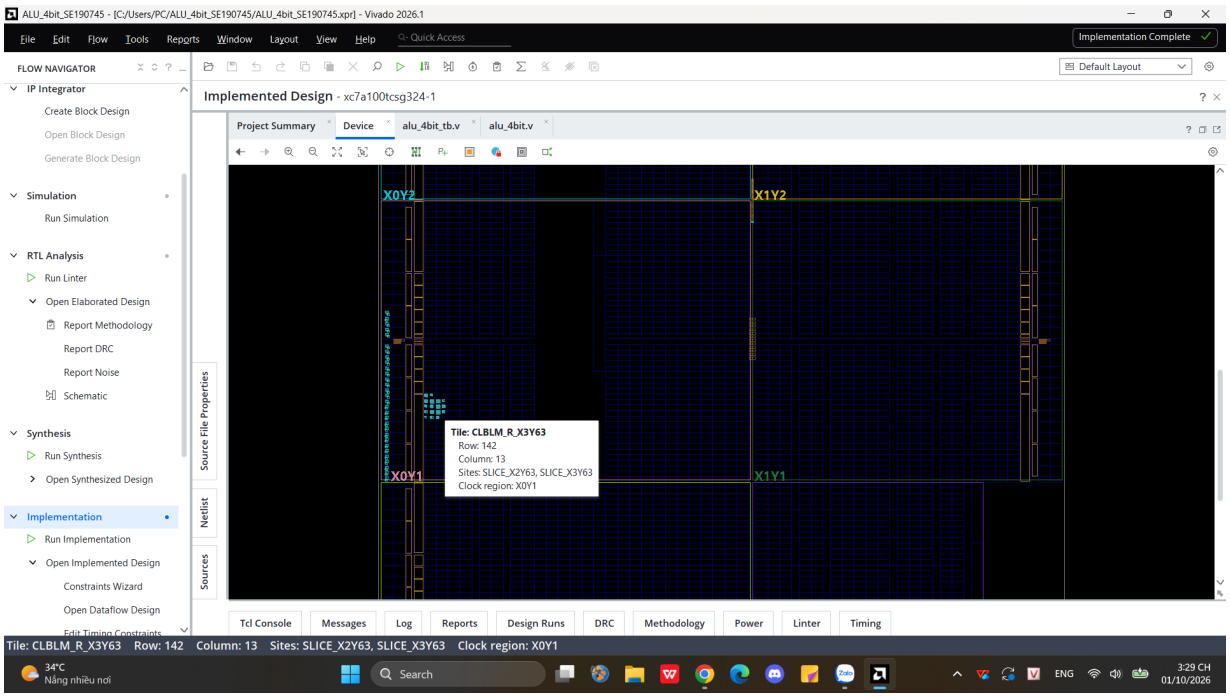


Figure 6: **Flow Screenshot F6:** Full-window Implemented Design Device floorplan showing physical placement inside Clock Region `X0Y1` at tile `CLBLM_R_X3Y63` (`SLICE_X2Y63`, `SLICE_X3Y63`).

4 Conclusion

The `ALU_4bit_SE190745` project successfully fulfills all design, self-checking verification, and physical implementation requirements in Vivado 2026.1. By sharing the 4-bit ripple-carry adder/subtractor across combinational and sequential execution modes, the circuit achieves 100% functional accuracy (`error_count = 0`) across all 10 opcodes and divide-by-zero exceptions while occupying only 60 LUTs and 26 Flip-Flops on the Artix-7 FPGA.